

Backend Architecture — Defence & Design Rationale

K Orchids · Project Green — Node.js + Express + PostgreSQL. A **modular monolith** organised by business domain, with a strict layered flow. The goal: a codebase a five-person team can build in parallel, reason about quickly, and trust under load.

1 · The structure — and why

```
apps/api/src/
  index.js           app bootstrap
  config/            db, env, mailer, cloundinary
  middleware/        auth, rbac, audit,
                    validate, errors, ratelimit
  utils/            money, stateMachine,
                    outbox, time, pagination
  jobs/             cron (outbox, aging...)
  modules/<domain>/
    .routes.js       HTTP + RBAC guards
    .controller.js   req/res only
    .service.js       business logic + txn
    .repository.js   SQL only
    .schema.js       Zod validation
    .test.js         tests
```

Why a modular monolith (not microservices). One deployable, one database, one transaction boundary — so an order approval can reserve stock, write the ledger, create the invoice and enqueue an email **atomically**. Microservices would scatter that across network calls and distributed transactions we don't need at this scale. **right-sized**

Why domain modules. Each of the ~20 domains (auth, orders, invoices...) is a self-contained folder. Five developers worked different modules in parallel with almost no file collisions — the folder *is* the contribution boundary.

Why the same six files everywhere. Convention over configuration: open any module and you already know where the HTTP, logic, SQL, validation and tests live. New features are predictable, not creative.

2 · Design fundamentals we applied

PRINCIPLE	HOW IT SHOWS UP
Layered separation of concerns	Strict one-way flow routes → controller → service → repository. Controllers never touch SQL; repositories never hold business rules. Each layer is testable in isolation.
Single responsibility / thin edges	Controllers only translate HTTP; logic lives in services; SQL is quarantined in repositories. Swapping HTTP for a queue would touch only the edge.
Validate at the boundary	Zod schemas reject malformed input before it reaches logic; a single validate() middleware maps failures to a consistent 422.
Consistent error contract	One error envelope {code,message,details,requestId} — the frontend handles every failure the same way; request-ids make logs traceable.
Transactions own invariants	Multi-write operations run in one tx() with SELECT ... FOR UPDATE; the audit row and outbox email commit <i>with</i> the change, never after.
Explicit state machines	A shared stateMachine util guards order/RFQ/invoice/RMA/delivery transitions, with status names matching the DB CHECK lists exactly.
Reliability patterns	Outbox for at-least-once email; idempotency keys on payments; append-only ledgers; cron jobs are lazy-initialised and env-guarded.
Security as middleware	Auth → RBAC (requirePermission) → audit are cross-cutting middleware, applied by declaration on each route, not re-implemented per handler.

3 · Simplicity — the choices we deliberately did *not* over-engineer

- **Raw pg over a heavy ORM.** SQL is explicit and reviewable — no hidden N+1s, no migration surprises, full control of FOR UPDATE and CTEs where correctness matters. The cost (writing SQL) is contained inside repositories.
- **Modular monolith over microservices.** Fewer moving parts, one transaction, trivial local setup. We kept clean seams so it *could* be split later — but only if scale ever demands it (YAGNI).
- **Boring, proven tech.** Express + PostgreSQL + node-cron — well-documented, easy to hire for, nothing exotic to debug at 2 a.m.
- **Convention over framework magic.** The six-file module pattern means no DI container, no decorators, no metaclass cleverness — just plain functions that are easy to read and step through.
- **Push correctness down to the database.** Constraints, unique keys and triggers mean the app stays simple — it doesn't have to defensively re-check what the schema already guarantees.

4 · Can we do better? — an honest critique

LIMITATION TODAY	WHAT WE'D DO WITH MORE TIME
Repetitive SQL across repositories (similar list/paginate/filter blocks).	Extract a small query helper / lightweight builder for the common list-filter-paginate shape, keeping raw SQL for the complex paths.
JavaScript, not typed. Bugs like column-name drift were caught late.	Adopt TypeScript (or JSDoc types) + generated DB types so schema/code mismatches fail at build time, not runtime.
Cross-module require() in a few services creates hidden coupling.	Define explicit module interfaces / a thin service registry so dependencies are declared, not reached into.
Tests are integration-leaning and not exhaustive.	Raise coverage with unit tests per service + contract tests , and wire them into the CI gate.
No auto-generated API docs.	Generate OpenAPI from the Zod schemas — one source of truth for validation and docs.
Single Postgres, in-process cache.	Add a read replica + Redis cache for report-heavy endpoints if traffic grows; the 60 s cache is a stop-gap.
Observability is basic (logs + request-ids).	Add structured metrics + tracing (e.g. OpenTelemetry) to watch the hot paths in production.

The trade-off we stand by. For a 7-week, five-person build of a correctness-critical trade platform, we optimised for **clarity, parallel teamwork and data integrity** over architectural sophistication. Every "do better" above is an *additive* improvement on a clean base — none requires unwinding a wrong decision.

One-line defence. The backend is intentionally **boring and layered**: thin HTTP edges, business logic in services, SQL in repositories, invariants in transactions and in the database — simple to read, safe under concurrency, and easy for five people to build together.